WEEK-6: User Account XML: Create an XML structure to store user account information, including details like username, password (encrypted), email, and purchase history.

1.The XML File (users.xml)

```
<?xml version="1.0" encoding="UTF-8"?>
<userAccounts>
   <user id="u1">
      <username>alice_smith</username>
<password>5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8</pass
word>
      <email>alice@example.com</email>
      <purchaseHistory>
         <purchase>
            <itemName>Wireless Keyboard</itemName>
            <price>$45.00</price>
            <date>2026-03-15</date>
         </purchase>
         <purchase>
            <itemName>USB-C Hub</itemName>
            <price>$25.00</price>
            <date>2026-04-02</date>
         </purchase>
      </purchaseHistory>
   </user>
   <user id="u2">
      <username>bob_jones</username>
<password>8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c92</passwo
rd>
      <email>bob.j@example.com</email>
      <purchaseHistory>
         <purchase>
            <itemName>Ergonomic Mouse</itemName>
            <price>$60.00</price>
            <date>2026-04-05</date>
         </purchase>
      </purchaseHistory>
   </user>
</userAccounts>
```

2. Page 1: User Directory (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>User Directory</title>
   <style>
      body { font-family: Arial, sans-serif; padding: 20px; background: #f4f4f9; }
      .user-card { background: white; padding: 15px; margin-bottom: 10px; border-radius: 5px;
box-shadow: 0 2px 4px rgba(0,0,0,0.1); }
      a { text-decoration: none; color: white; background: #007bff; padding: 5px 10px; border-
radius: 3px; font-size: 14px; }
   </style>
</head>
<body>

   <h2>User Accounts Dashboard</h2>
   <div id="users-container">Loading users...</div>

   <script>
      fetch('users.xml')
         .then(response => response.text())
         .then(data => {
            const parser = new DOMParser();
            const xml = parser.parseFromString(data, "application/xml");
            const users = xml.querySelectorAll('user');
            let output = '';

            users.forEach(user => {
               const id = user.getAttribute('id');
               const username = user.querySelector('username').textContent;
               const email = user.querySelector('email').textContent;

               output += `
                  <div class="user-card">
                     <h3>${username}</h3>
                     <p>Email: ${email}</p>
                     <a href="history.html?id=${id}">View Purchase History</a>
                  </div>
               `;
            });
            document.getElementById('users-container').innerHTML = output;
         });
   </script>
```

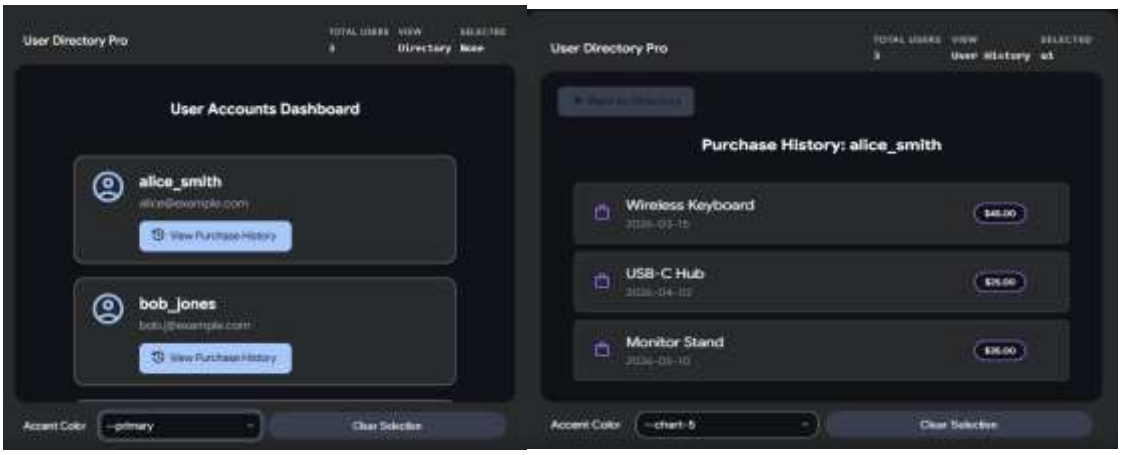
```
</body>
</html>
```

3. Page 2: Purchase History (history.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <title>Purchase History</title>
   <style>
      body { font-family: Arial, sans-serif; padding: 20px; background: #f4f4f9; }
      .purchase { background: white; padding: 10px; margin-bottom: 5px; border-left: 4px solid
#28a745; }
      a.back { display: inline-block; margin-bottom: 15px; color: #007bff; text-decoration: none; }
   </style>
</head>
<body>

   <a href="index.html" class="back">← Back to Directory</a>
   <h2 id="user-title">Purchase History</h2>
   <div id="history-container">Loading...</div>

   <script>
      // Get the user ID from the URL (e.g., ?id=u1)
      const params = new URLSearchParams(window.location.search);
      const userId = params.get('id');

      fetch('users.xml')
         .then(response => response.text())
         .then(data => {
            const parser = new DOMParser();
            const xml = parser.parseFromString(data, "application/xml");

            // Find the specific user by ID
            const user = xml.querySelector(`user[id="${userId}"]`);

            if (!user) {
               document.getElementById('history-container').innerHTML = "<p>User not
found.</p>";
               return;
            }

            const username = user.querySelector('username').textContent;
```

```
            document.getElementById('user-title').textContent =`Purchase History for ${username}`;
            const purchases = user.querySelectorAll('purchaseHistory purchase');
            let output = '';

            if (purchases.length === 0) {
               output = "<p>No purchases found.</p>";
            } else {
               purchases.forEach(p => {
                  const item = p.querySelector('itemName').textContent;
                  const price = p.querySelector('price').textContent;
                  const date = p.querySelector('date').textContent;

                  output += `
                     <div class="purchase">
                        <strong>${item}</strong> - ${price} <br>
                        <small>Purchased on: ${date}</small>
                     </div>
                  `;
               });
            }
            document.getElementById('history-container').innerHTML = output;
         });
   </script>
</body>
</html>
```

OUTPUT

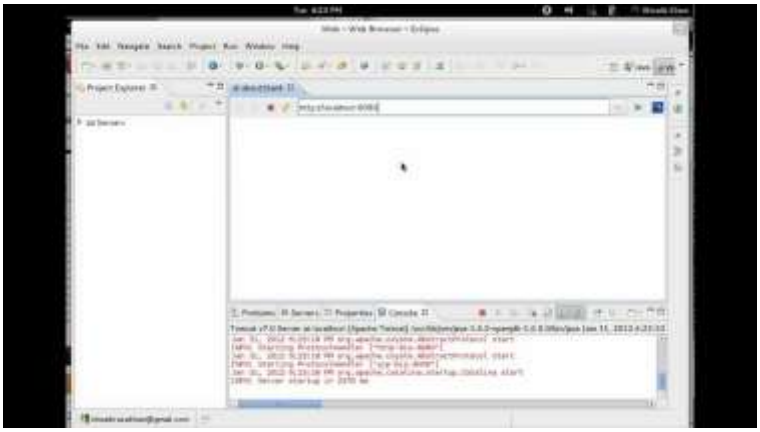


DATE: EXERCISE NO: EXERCISE NAME:

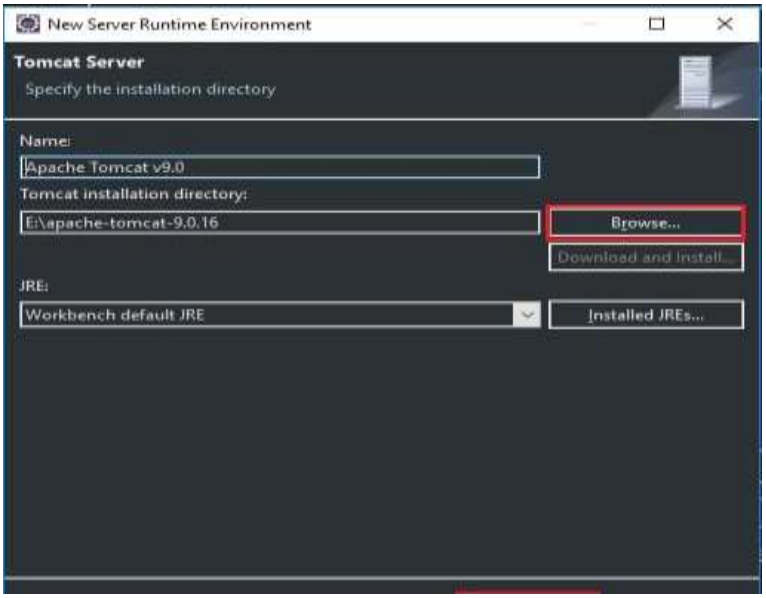
WEEK-7: Install and configure Apache Tomcat server in Eclipse

Step 1: Install Apache Tomcat (outside Eclipse)

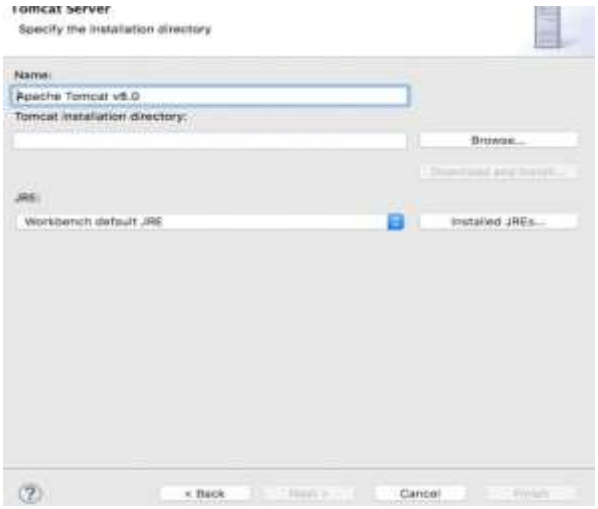
Step 2: Open Eclipse in Java EE Perspective



Step 3: Add Tomcat Server Runtime



Step 4: Create a Tomcat Server in Eclipse



Step 5: Start the Tomcat Server

Step 6: Create a Simple Web Project and Deploy on Tomcat

Step 7: Verify Everything is Working

WEEK-8: Servlet for Book Retrieval: Develop a servlet that retrieves book information from a database and sends it to the content frame.

1.Database Setup (SQL)

```
CREATE TABLE books (
   id INT PRIMARY KEY,
   title VARCHAR(100),
   author VARCHAR(100),
   year INT
);

INSERT INTO books VALUES (1, 'Clean Code', 'Robert C. Martin', 2008);
INSERT INTO books VALUES (2, 'The Pragmatic Programmer', 'Andrew Hunt', 1999);
INSERT INTO books VALUES (3, 'Design Patterns', 'Erich Gamma', 1994);
```

2. Main Layout (index.html)

```
<!DOCTYPE html>
<html>
<head>
   <title>Book Library</title>
</head>
<frameset cols="20%, 80%">
   <frame src="nav.html" name="navFrame">
   <frame src="content.html" name="contentFrame">
</frameset>
</html>
```

3. Navigation Frame (nav.html)

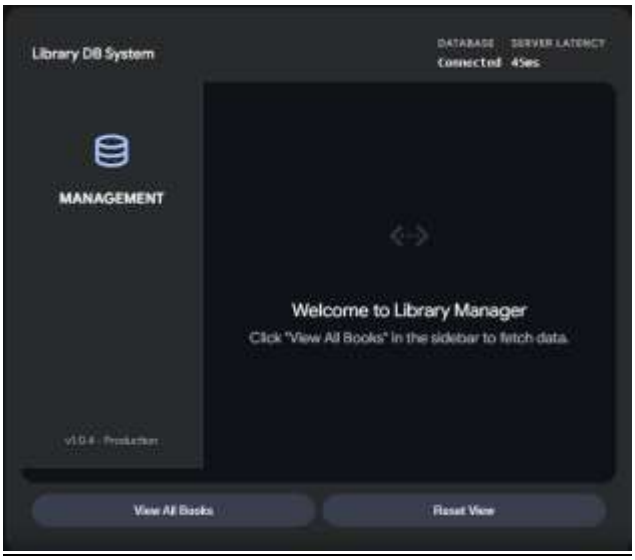
```
<!DOCTYPE html>
<html>
<body style="font-family: Arial, sans-serif; background-color: #f4f4f4;">
   <h3>Menu</h3>
   <a href="BookServlet" target="contentFrame">View All Books</a>
</body>
</html>
```

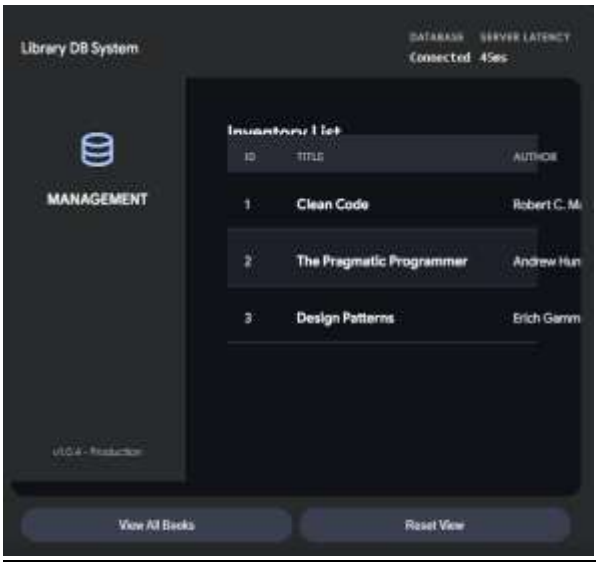
4. The Servlet (BookServlet.java)

```
import java.io.*;
import java.sql.*;
import javax.servlet.*;
import javax.servlet.http.*;
public class BookServlet extends HttpServlet {
   public void doGet(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {
      response.setContentType("text/html");
      PrintWriter out = response.getWriter();
      out.println("<html><head><style>table {width: 100%; border-collapse: collapse;} th, td
{border: 1px solid black; padding: 8px; text-align: left;} th {background-color:
#f2f2f2;}</style></head><body>");
      out.println("<h2>Book List</h2>
      try {
         // 1. Load Driver (Example for MySQL)
         Class.forName("com.mysql.cj.jdbc.Driver");

         // 2. Connect to DB (Change URL, username, password!)
         Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/yourdb",
"root", "password");

         // 3. Execute Query
         Statement stmt = con.createStatement();
         ResultSet rs = stmt.executeQuery("SELECT * FROM books");

         // 4. Print HTML Table
      out.println("<table><tr><th>ID</th><th>Title</th><th>Author</th><th>Year</th></tr>");
         while (rs.next()) {
            out.println("<tr>");
            out.println("<td>" + rs.getInt("id") + "</td>");
            out.println("<td>" + rs.getString("title") + "</td>");
            out.println("<td>" + rs.getString("author") + "</td>");
            out.println("<td>" + rs.getInt("year") + "</td>");
            out.println("</tr>");
         }
         out.println("</table>");

         con.close();
      } catch (Exception e) {
         out.println("<p>Error: " + e.getMessage() + "</p>");
      }

      out.println("</body></html>");
```

```
    }
}
```

OUTPUT


Library DB System
DATABASE SERVER LATENCY
Connected 45ms
MANAGEMENT
Welcome to Library Manager
Click "View All Books" in the sidebar to fetch data.
View All Books
Reset View


Library DB System
DATABASE SERVER LATENCY
Connected 45ms
MANAGEMENT
ID
TITLE
AUTHOR
1
Clean Code
2
The Pragmatic Programmer
3
Design Patterns
View All Books
Reset View

WEEK-9: User Authentication: Implement user authentication using servlets and JSP. Create a login page and restrict access to certain pages for authenticated users.

1.The Login Page (login.jsp)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
   <title>Login</title>
   <style>
      body { font-family: Arial, sans-serif; background: #f4f4f9; display: flex; justify-content: center; padding-top: 50px; }
      .login-box { background: white; padding: 20px; border-radius: 5px; box-shadow: 0 0 10px rgba(0,0,0,0.1); width: 300px; }
      input[type="text"], input[type="password"] { width: 90%; padding: 10px; margin: 10px 0; }
      button { width: 100%; padding: 10px; background: #007bff; color: white; border: none; cursor: pointer; }
      .error { color: red; font-size: 14px; }
   </style>
</head>
<body>
   <div class="login-box">
      <h2>Sign In</h2>
      <% if(request.getAttribute("error") != null) { %>
         <p class="error"><%= request.getAttribute("error") %></p>
      <% } %>
      <form action="LoginServlet" method="post">
         <input type="text" name="username" placeholder="Username" required><br>
         <input type="password" name="password" placeholder="Password" required><br>
         <button type="submit">Login</button>
      </form>
   </div>
</body>
</html>
```

2. The Authentication Servlet (LoginServlet.java)

```
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {
```

```
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        String user = request.getParameter("username");
        String pass = request.getParameter("password");
        // Hardcoded check (Replace with DB validation in production)
        if ("admin".equals(user) && "password123".equals(pass)) {
            // Create a session and store the username
            HttpSession session = request.getSession();
            session.setAttribute("username", user);
            // Redirect to the restricted page
            response.sendRedirect("dashboard.jsp");
        } else {
            // Send back to login page with an error
            request.setAttribute("error", "Invalid username or password");
            request.getRequestDispatcher("login.jsp").forward(request, response);
        }
    }
}
```

3. The Restricted Page (dashboard.jsp)

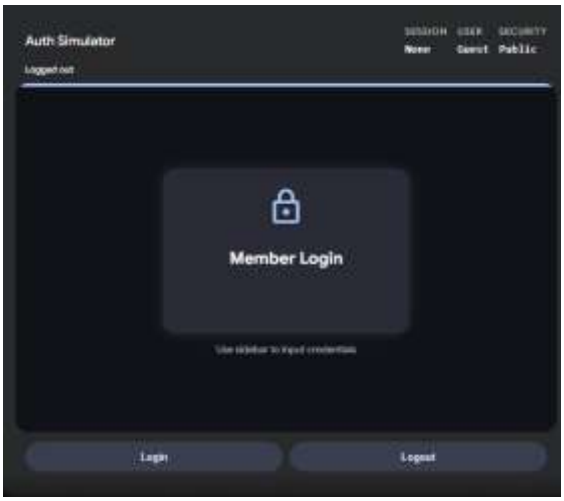
```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%
    // Security Check: Restrict access to authenticated users only
    if (session.getAttribute("username") == null) {
        response.sendRedirect("login.jsp");
        return; // Stop rendering the page
    }
%>
<!DOCTYPE html>
<html>
<head>
    <title>Dashboard - Restricted Area</title>
    <style>
        body { font-family: Arial, sans-serif; padding: 20px; text-align: center; }
        .card { background: #e0f7fa; padding: 20px; display: inline-block; border-radius: 8px; }
        a.logout { color: white; background: #dc3545; padding: 8px 15px; text-decoration: none; border-radius: 4px; }
    </style>
</head>
<body>
    <div class="card">
        <h2>Welcome, <%= session.getAttribute("username") %>!</h2>
        <p>You have successfully accessed the restricted dashboard.</p>
```

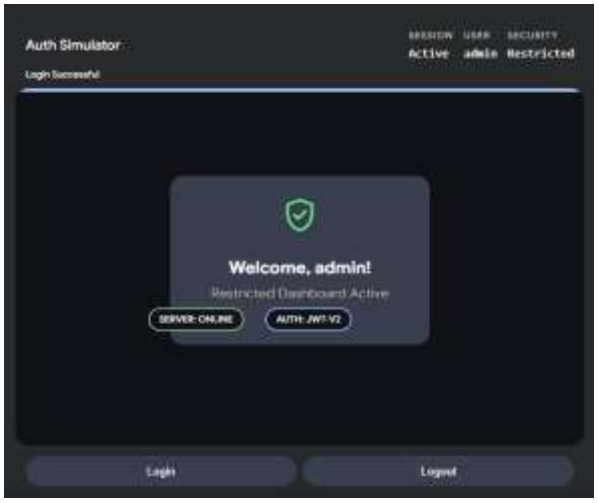
```
        <br>
        <a href="LogoutServlet" class="logout">Logout</a>
    </div>
</body>
</html>
```

4. The Logout Servlet (LogoutServlet.java)

```
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/LogoutServlet")
public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        HttpSession session = request.getSession(false); // Get existing session
        if (session != null) {
            session.invalidate(); // Destroy the session
        }
        response.sendRedirect("login.jsp");
    }
}
```

OUTPUT





WEEK-10:
Hibernate: Hibernate Entity Mapping: Create Hibernate entity classes to represent books and users. Map these entities to corresponding database tables

1.The Book Entity (Book.java)

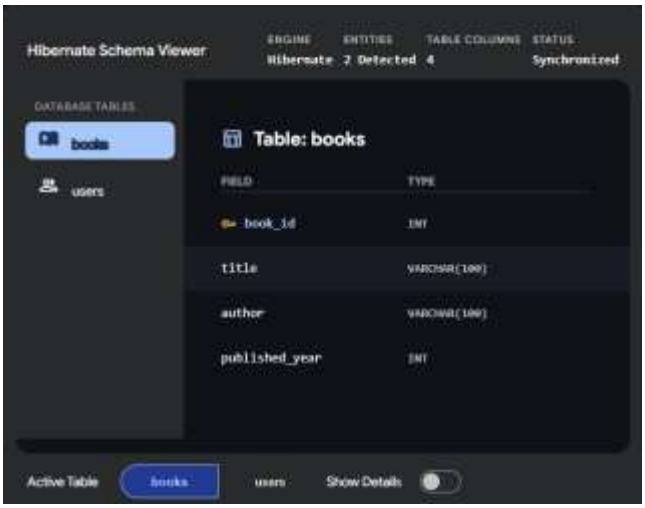
```
import javax.persistence.*;
@Entity
@Table(name = "books")
public class Book {
   @Id // Marks this as the Primary Key
   @GeneratedValue(strategy = GenerationType.IDENTITY) // Auto-increments the ID
   @Column(name = "book_id")
   private int id;
   @Column(name = "title", nullable = false, length = 100)
   private String title;
   @Column(name = "author", length = 100)
   private String author;
   @Column(name = "published_year")
   private int year;
   // Default constructor is required by Hibernate
   public Book() {}
   // Getters and Setters
   public int getId() { return id; }
   public void setId(int id) { this.id = id; }
   public String getTitle() { return title; }
   public void setTitle(String title) { this.title = title; }
   public String getAuthor() { return author; }
   public void setAuthor(String author) { this.author = author; }
   public int getYear() { return year; }
   public void setYear(int year) { this.year = year; }
}
```

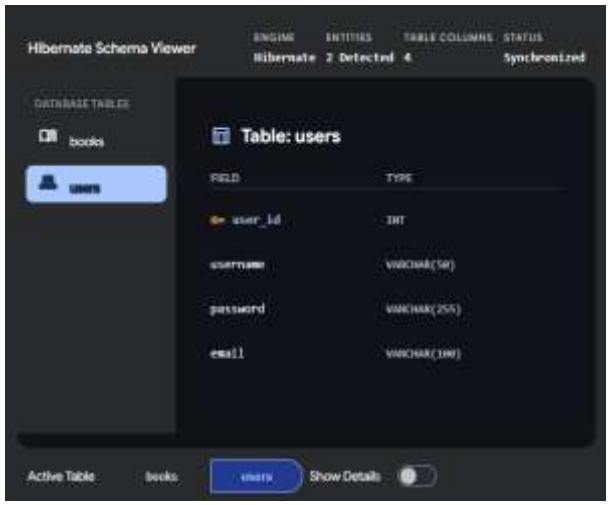
2. The User Entity (User.java)

```
import javax.persistence.*;
@Entity
@Table(name = "users")
public class User {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   @Column(name = "user_id")
   private int id;
   @Column(name = "username", unique = true, nullable = false, length = 50)
   private String username;
```

```
    @Column(name = "password", nullable = false) // Passwords should ideally be hashed
    private String password;
    @Column(name = "email", length = 100)
    private String email;
    // Default constructor is required by Hibernate
    public User() {}
    // Getters and Setters
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
}
```

OUTPUT





WEEK-11:
CRUD Operations: Implement CRUD operations using Hibernate for managing book information in the database.

1.The Database Helper (HibernateUtil.java)

```
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class HibernateUtil {
    private static final SessionFactory sessionFactory;
    static {
        try {
            // Loads configuration and builds the session factory
            sessionFactory = new Configuration().configure().buildSessionFactory();
        } catch (Throwable ex) {
            System.err.println("Initial SessionFactory creation failed." + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }

    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}
```

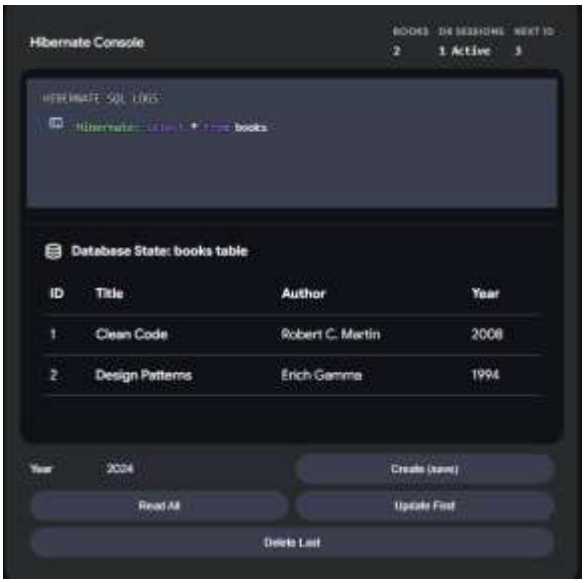
2. The CRUD Implementation (BookDAO.java)

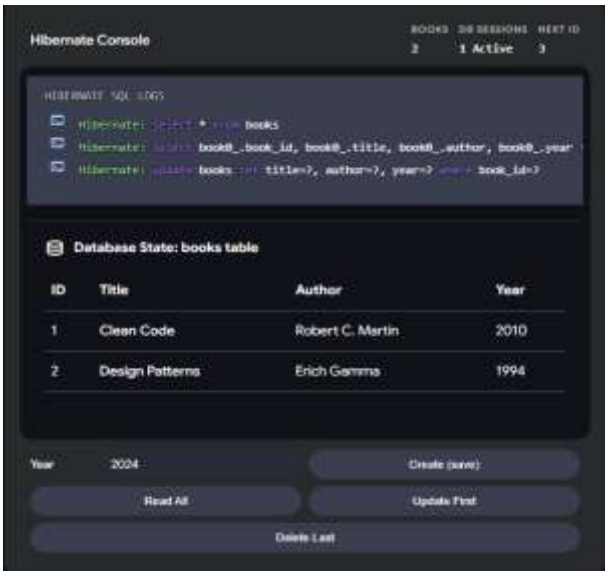
```
import org.hibernate.Session;
import org.hibernate.Transaction;
import java.util.List;
public class BookDAO {
    // -------------------------------------------------------
    // CREATE: Add a new book to the database
    // -------------------------------------------------------
    public void saveBook(Book book) {
        Transaction transaction = null;
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();
            session.save(book); // Hibernate automatically writes the INSERT SQL
            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) {
                transaction.rollback(); // Undo changes if something goes wrong
            }
            e.printStackTrace();
```

```
        }
    }

    // ---------------------------------------------------------
    // READ: Get a specific book by its ID
    // ---------------------------------------------------------
    public Book getBook(int id) {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            return session.get(Book.class, id); // Hibernate writes the SELECT SQL
        }
    }

    // ---------------------------------------------------------
    // READ: Get a list of all books in the database
    // ---------------------------------------------------------
    @SuppressWarnings("unchecked")
    public List<Book> getAllBooks() {
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            // HQL (Hibernate Query Language) uses the Java class name, not the DB table name
            return session.createQuery("from Book").list();
        }
    }

    // ---------------------------------------------------------
    // UPDATE: Modify an existing book's details
    // ---------------------------------------------------------
    public void updateBook(Book book) {
        Transaction transaction = null;
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();
            session.update(book); // Hibernate writes the UPDATE SQL
            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) {
                transaction.rollback();
            }
            e.printStackTrace();
        }
    }

    // ---------------------------------------------------------
    // DELETE: Remove a book from the database
    // ---------------------------------------------------------
    public void deleteBook(int id) {
        Transaction transaction = null;
```

```
        try (Session session = HibernateUtil.getSessionFactory().openSession()) {
            transaction = session.beginTransaction();

            // First, fetch the book we want to delete
            Book book = session.get(Book.class, id);
            if (book != null) {
                session.delete(book); // Hibernate writes the DELETE SQL
            }

            transaction.commit();
        } catch (Exception e) {
            if (transaction != null) {
                transaction.rollback();
            }
            e.printStackTrace();
        }
    }
}
```

OUTPUT





WEEK-12: Full-Stack Integration: Integrate JavaScript with Servlets: Enhance the shopping cart functionality by integrating JavaScript with servlets. Allow users to update the cart without page refresh.

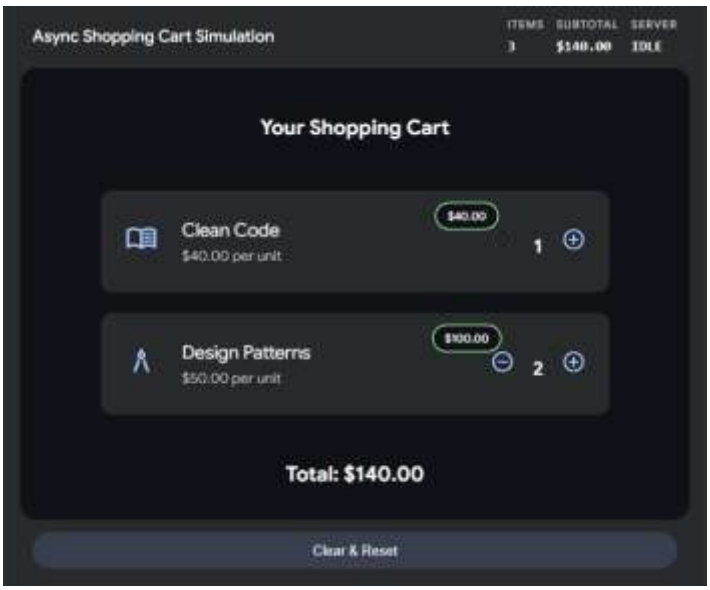
1.The Frontend (cart.jsp)

```
<!DOCTYPE html>
<html>
<head>
   <title>AJAX Shopping Cart</title>
   <style>
      body { font-family: Arial, sans-serif; padding: 20px; max-width: 500px; }
      .cart-item { padding: 10px; border-bottom: 1px solid #ddd; display: flex; justify-content:
space-between; align-items: center; }
      .cart-item input { width: 50px; padding: 5px; }
      .status-message { color: #28a745; font-size: 14px; display: none; margin-bottom: 10px; }
      .total-row { margin-top: 20px; font-size: 1.2em; font-weight: bold; text-align: right; }
   </style>
</head>
<body>

   <h2>Your Shopping Cart</h2>
   <div id="updateStatus" class="status-message">Cart updated successfully!</div>

   <div class="cart-item">
      <span>Clean Code ($40.00)</span>
      <input type="number" value="1" min="1" onchange="updateCart(1, 40.00, this.value)">
   </div>
   <div class="cart-item">
      <span>Design Patterns ($50.00)</span>
      <input type="number" value="1" min="1" onchange="updateCart(2, 50.00, this.value)">
   </div>

   <div class="total-row">
      Total: $<span id="cartTotal">90.00</span>
   </div>

   <script>
      // JavaScript for asynchronous updating (AJAX)
      function updateCart(bookId, price, quantity) {
         // Send a background request to the Servlet
         fetch('CartServlet', {
            method: 'POST',
            headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
```

```
            body: `bookId=${bookId}&price=${price}&quantity=${quantity}`
        })
        .then(response => response.json()) // The Servlet will return JSON data
        .then(data => {
            // Update the total on the screen dynamically
            document.getElementById('cartTotal').innerText = data.newTotal;

            // Flash a success message
            let status = document.getElementById('updateStatus');
            status.style.display = 'block';
            setTimeout(() => { status.style.display = 'none'; }, 2000); // Hide after 2 seconds
        })
        .catch(error => console.error('Error updating cart:', error));
    }
  </script>
</body>
</html>
```

2. The Backend (CartServlet.java)

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/CartServlet")
public class CartServlet extends HttpServlet {
   protected void doPost(HttpServletRequest request, HttpServletResponse response)
          throws ServletException, IOException {
      // 1. Read the parameters sent by Javascript
      int bookId = Integer.parseInt(request.getParameter("bookId"));
      double price = Double.parseDouble(request.getParameter("price"));
      int quantity = Integer.parseInt(request.getParameter("quantity"));
      // 2. Business Logic: Update session cart and recalculate total
      // Note: For this isolated example, we are just calculating the total of the changed item.
      // In a real app, you would retrieve the user's cart object from the session and sum all items.
      HttpSession session = request.getSession();
      // Simulating a cart total calculation (Assuming they only have this one item type for the demo
math)
      // You would replace this with actual cart iteration logic.
      double simulatedNewTotal = price * quantity;
      // 3. Set the response content type to JSON
      response.setContentType("application/json");
      response.setCharacterEncoding("UTF-8");

      // 4. Send the JSON response back to the Javascript fetch() call
```

```
        PrintWriter out = response.getWriter();
        out.print("{\"newTotal\": \"" + String.format("%.2f", simulatedNewTotal) + "\"}");
        out.flush();
    }
}
```

OUTPUT